Universidad de los Andes

Departamento de Ingeniería Industrial · Modelado de Sistemas bajo Incertidumbre

La Dinámica de los Eventos Discretos

Alejandra Tabares Pozos

Resumen

Esta lectura disecciona la arquitectura interna de un modelo de Simulación de Eventos Discretos (DES). Se explican los mecanismos de avance del tiempo, con énfasis en el enfoque de "Próximo Evento", y se detalla la estructura de la Lista de Eventos Futuros (FEL). Finalmente, se presenta una traza manual completa (Hand Simulation) de un sistema de colas para ilustrar la actualización de variables de estado y contadores estadísticos.

1. Definición de Sistemas de Eventos Discretos

Un sistema de eventos discretos es aquel en el que las variables de estado cambian solo en puntos discretos en el tiempo. Estos puntos se denominan **eventos**. Un evento se define como una ocurrencia instantánea que puede cambiar el estado del sistema.

Entre dos eventos consecutivos, no ocurre ningún cambio en el sistema; el sistema permanece "congelado". Esta propiedad es fundamental para la eficiencia computacional de la simulación.

Conceptos Clave:

- **Reloj de Simulación (CLOCK):** Una variable que almacena el valor actual del tiempo simulado. No tiene relación con el tiempo real de ejecución de la computadora.
- **Variables de Estado:** Colección mínima de variables necesarias para describir el sistema en un tiempo t (ej. número de clientes en cola $L_q(t)$, estado del servidor $L_s(t)$).
- **Contadores Estadísticos:** Variables que acumulan datos para el análisis de salida (ej. total de espera, número de clientes atendidos).

2. Mecanismos de Avance del Tiempo

Existen dos enfoques principales para avanzar el reloj de simulación:

2.1. Incremento de Tiempo Fijo (Fixed-Increment)

El reloj avanza en intervalos constantes Δt. Después de cada actualización ($t \to t + \Delta t$), se verifica si ha ocurrido algún evento durante ese intervalo.

- *Desventaja:* Si Δt es pequeño, se desperdician muchos ciclos de CPU verificando intervalos donde no pasa nada. Si Δt es grande, se pierde precisión en la ocurrencia exacta de los eventos.
- *Uso:* Común en sistemas continuos (Dinámica de Sistemas) o simulaciones de tiempo real, pero ineficiente para eventos discretos.

2.2. Avance al Próximo Evento (Next-Event Time Advance)

Este es el mecanismo estándar en la simulación moderna. 1. El reloj se inicializa en $t = 0$. 2. Se determinan los tiempos de ocurrencia de los eventos futuros (almacenados en la FEL). 3. El reloj **salta** inmediatamente desde el tiempo actual t hasta el tiempo del evento más inminente t_{min}. 4. Se actualiza el estado del sistema según la lógica de ese evento. 5. Se repite el proceso.

Los periodos de inactividad se saltan instantáneamente, lo que permite simular años de operación en segundos.

3. La Lista de Eventos Futuros (FEL)

La **Future Event List (FEL)** es la estructura de datos central del motor de simulación. Es una lista ordenada (ranking) de todos los eventos que se sabe que ocurrirán, ordenados cronológicamente por su tiempo de ocurrencia.

La FEL contiene "avisos de eventos" (event notices). Cada aviso tiene al menos dos datos:

1. **Tiempo del evento:** Cuándo ocurrirá.
2. **Tipo de evento:** Qué sucederá (ej. "Llegada", "Salida", "Fin de Simulación").

La dinámica es cíclica: procesar el evento al tope de la lista (head) a menudo genera nuevos eventos que se insertan en la FEL en su posición correcta.

4. Simulación Manual (The Hand Simulation)

Para comprender la interacción entre el Reloj, la FEL y las variables de estado, realizaremos una simulación paso a paso de un sistema de colas simple con un solo servidor (Single Server Queue) .

4.1. Escenario

- **Variables de Estado:** $LQ(t)$ (longitud de cola), $LS(t)$ (estado servidor: 0=Ocioso, 1=Ocupado).
- **Entradas (Determinísticas para este ejercicio):**
  - Tiempos entre llegadas (Interarrival Times - IA): $A_1 = 2, A_2 = 4, A_3 = 3$. (Primera llegada

en $t = 0$).

- Tiempos de servicio (Service Times - S): $S_1 = 3, S_2 = 3, S_3 = 2$.

- **Eventos Posibles:** (A) Llegada de cliente, (D) Salida de cliente.

4.2. Traza Paso a Paso

Paso	**Reloj** (t)	**LQ**	**LS**	**Lista de Eventos Futuros (FEL)** *[(Tipo, Tiempo)]*	**Comentarios y Acciones**
0	0	0	0	[(A, 0)]	**Inicialización.** Se programa la primera llegada.
1	0	0	1	[(D, 3), (A, 2)]	**Evento: Llegada (Cliente 1).** 1. Reloj avanza a 0. 2. Servidor ocioso → Pasa a atender. LS=1. 3. Generar Salida: $t + S_1 = 0 + 3 = 3$. 4. Generar Prox. Llegada: $t + IA_1 = 0 + 2 = 2$.
2	2	1	1	[(D, 3), (A, 6)]	**Evento: Llegada (Cliente 2).** 1. Reloj avanza a 2. 2. Servidor ocupado (LS=1) → A cola. LQ=1. 3. Generar Prox. Llegada: $t + IA_2 = 2 + 4 = 6$. *Nota: El evento (D, 3) sigue pendiente.
3	3	0	1	[(A, 6), (D, 6)]	**Evento: Salida (Cliente 1).** 1. Reloj avanza a 3. 2. Hay cola (LQ=1) → Cliente 2 entra a servicio. 3. LQ decrementar a 0. LS sigue en 1. 4. Generar Salida C2: $t + S_2 = 3 + 3 = 6$.
4	6	1	1	[(D, 6)*, (A, 9)]	**Evento: Llegada (Cliente 3).** *Empate en tiempo (6). Arbitrariamente procesamos 1. Reloj avanza a 6. 2. Servidor ocupado → Cola. LQ=1. 3. Generar Prox. Llegada: $t + IA_3 = 6 + 3 = 9$.
5	6	0	1	[(A, 9), (D, 8)]	**Evento: Salida (Cliente 2).** 1. Reloj en 6 (sin cambio). 2. Cliente 3 entra a servicio. LQ=0. 3. Generar Salida C3: $t + S_3 = 6 + 2 = 8$.

Tabla 1: Traza manual de simulación (M/G/1)

Análisis de la Traza: Observe el paso 4. Ocurren dos eventos simultáneamente en $t = 6$. En la lógica computacional, debemos tener reglas de desempate (prioridades). En este caso, procesamos la llegada antes que la salida, pero el orden podría afectar momentáneamente las estadísticas de longitud máxima de cola.

5. Implementación

A continuación, implementaremos un motor de simulación básico (bare-bones) que replica la lógica de la FEL y el avance de tiempo, sin utilizar librerías de simulación externas. Esto demuestra cómo se programa la estructura subyacente.

6. Conclusión

Comprender la manipulación de la FEL es vital. Cuando un software comercial arroja errores o resultados extraños, casi siempre se debe a una mala lógica en la programación de eventos (ej. bucles infinitos de eventos en tiempo cero). La simulación manual, aunque tediosa, revela la contabilidad exacta que valida nuestros modelos computacionales. Es una herramienta valiosa para validar la lógica que hemos usado para crear nuestra simulación.